Mastering Observability

Logging, Serilog, and Seq in ASP.NET Core

1. What is Logging and Why Do We Need It?

Imagine flying a commercial airliner without a dashboard or a black box. If an engine fails, you wouldn't know which one, why it failed, or when it started failing. In the software world, logging is your application's **black box and dashboard combined**.

When an application runs in production, you cannot attach a debugger. Logging provides:

- **Troubleshooting:** Exact stack traces and variable states when a 500 Server Error occurs.
- **Auditing:** Security records of who did what and when (e.g., "User X modified Order Y").
- **Observability:** The ability to track the health, performance, and flow of complex systems (like microservices) from the outside.

2. Logging in ASP.NET Core (The Basics)

ASP.NET Core has a built-in logging abstraction centered around the `ILogger<T>` interface. The `<T>` is the "Category" (usually the class name), which automatically tags your logs so you know exactly which file generated them.

The 6 Logging Levels

Level	Use Case	Example
Trace (0)	Extreme detail, passwords, HTTP bodies.	*Rarely used in production.*
Debug (1)	Developer diagnostic info, loop variables.	`LogDebug("Loop {i}", i)`
Information (2)	Happy path milestones, business events.	`LogInformation("Order {Id} paid", id)`
Warning (3)	Handled exceptions, non-fatal behavior.	`LogWarning("Failed login")`
Error (4)	Current operation failed (e.g. DB timeout).	`LogError(ex, "Save failed")`
Critical (5)	App crashing, out of memory, disk full.	`LogCritical(ex, "DB offline")`

> 💡 **Senior Developer Tip: The Rule of Three**
>
> Don't overcomplicate it. In daily feature development, you will mostly use **Information** (business success), **Warning** (suspicious behavior/retries), and **Error** (caught exceptions).

3. The Paradigm Shift: Structured Logging

The biggest mistake junior developers make is treating logs like text strings. Modern enterprise logging treats logs as **Queryable Data**.

> ❌ **The Junior Way (String Interpolation)**
>
> `_logger.LogInformation($"Order {orderId} processed for {userId}");`
>
> This creates a flat text string: "Order 123 processed for 456". You cannot easily query this in a database.

> ✅ **The Senior Way (Message Templates)**
> `_logger.LogInformation("Order {OrderId} processed for {UserId}", orderId, userId);`
> This creates a JSON object where `OrderId` and `UserId` are preserved as separate variables. In your log dashboard, you can literally search: `SELECT * WHERE UserId = 456`.

4. The Dream Team: Serilog and Seq

While Microsoft provides the `ILogger` interface (the steering wheel), we need a powerful engine to drive the logs and a beautiful dashboard to view them.

Serilog (The Producer)

Serilog is a third-party Logging Provider built specifically for Structured Logging. It intercepts your `ILogger` calls, converts them to rich JSON, and routes them to "Sinks" (Console, Files, Cloud, Seq).

Seq (The Consumer)

Seq is a centralized log server and search dashboard. Instead of reading boring text files on a Linux server, your application sends its JSON logs to Seq over HTTP. Seq gives you a UI to filter, graph, and set up alerts for your logs.

5. Putting It All Together (Configuration)

Step 1: Docker Compose Infrastructure

Run Seq alongside your microservices so they can talk to each other over the Docker network.

```
services:
  ops-seq:
    image: datalust/seq:latest
    environment:
      - ACCEPT_EULA=Y
      - SEQ_FIRSTRUN_ADMINPASSWORD=Password123!
    ports:
      - "5341:5341" # Ingestion port
      - "8081:80"   # UI port
    volumes:
      - seq_data:/data
```

Step 2: appsettings.json

Configure Serilog here so you can change settings dynamically without recompiling C# code.

```
{
  "Serilog": {
    "Using": [ "Serilog.Sinks.Console", "Serilog.Sinks.Seq" ],
    "MinimumLevel": {
      "Default": "Information",
      "Override": {
        "Microsoft.AspNetCore": "Warning" // Mute framework noise
      }
    },
    "WriteTo": [
      { "Name": "Console" },
      {
        "Name": "Seq",
        "Args": { "serverUrl": "http://localhost:5341" }
      }
    ],
    "Enrich": [ "FromLogContext", "WithMachineName" ],
    "Properties": { "Application": "OrderService" }
  }
}
```

Step 3: Program.cs Implementation

```
var builder = WebApplication.CreateBuilder(args);

// 1. Tell Host to use Serilog
builder.Host.UseSerilog((context, config) => {
    config.ReadFrom.Configuration(context.Configuration);
});

var app = builder.Build();

// 2. Condense HTTP request logs into a single, clean log entry
app.UseSerilogRequestLogging();

app.MapControllers();
app.Run();
```

6. What Else Should I Know? (Advanced Topics)

Here are three critical logging concepts that elevate your architecture to the highest standard.

A. Logging Scopes (`BeginScope`)

If you are processing an Order, you don't want to pass the `OrderId` into every single `LogInformation` call. By creating a scope, all logs generated within that block automatically get tagged with the variables.

```
using (_logger.BeginScope(new Dictionary<string, object> { ["OrderId"] = orderId }))
{
    _logger.LogInformation("Validating payment..."); // Auto-tagged with OrderId!
    _logger.LogInformation("Saving to DB...");       // Auto-tagged with OrderId!
}
```

B. Correlation IDs (Distributed Tracing)

In a microservice architecture, an HTTP request hits the API Gateway, goes to the Order Service, then to the Payment Service. How do you track *one* user's action across three different apps in Seq?

You generate a **Correlation ID** (a GUID) at the Gateway and pass it in the HTTP Headers (e.g., `X-Correlation-ID`). Every microservice reads this header and attaches it to its logs. In Seq, you simply search `CorrelationId = '...'` and you will see the full lifecycle of the request across all services!

C. High-Performance Logging (Source Generators)

Using `_logger.LogInformation()` requires memory allocation and boxing of variables. For extreme high-performance applications (like handling 10,000+ requests per second), Microsoft introduced **Compile-Time Logging Source Generators** in .NET 6+.

```
public partial class OrderService
{
    // The compiler writes the high-performance IL code for you!
    [LoggerMessage(Level = LogLevel.Information, Message = "Order {OrderId}
created")]
    public partial void LogOrderCreated(Guid orderId);
}
```

Instead of calling `_logger` directly, you call `LogOrderCreated(orderId)`. This uses zero memory allocation and is the fastest way to log in .NET.